

CHAPTER 4



Text Classification

Learning to process and understand text is one of the first steps on the journey to getting meaningful insights from textual data. Though it is important to understand how language is structured and specific text syntax patterns, that alone is not sufficient to be of much use to businesses and organizations who want to derive useful patterns and insights and get maximum use out of their vast volumes of text data. Knowledge of language processing coupled with concepts from analytics and machine learning (ML) help in building systems that can leverage text data and help solve real-world practical problems which benefit businesses.

Various aspects of ML include supervised learning, unsupervised learning, reinforcement learning, and more recently deep learning. Each of these concepts involves several techniques and algorithms that can be leveraged on text data and to build self-learning systems that do not need too much manual supervision. An ML model is a combination of data and algorithms—you got a taste of that in Chapter 3 as we built our own parsers and taggers. The benefit of ML is that once a model is trained, we can directly use it on new and previously unseen data to start seeing useful insights and desired results.

One of the most relevant and challenging problems is *text classification* or *categorization*, which involves trying to organize text documents into various categories based on inherent properties or attributes of each text document. This is used in various domains, including email spam identification and news categorization. The concept may seem simple, and if you have a small number of documents, you can look at each document and gain some idea about what it is trying to indicate. Based on this knowledge, you can group similar documents into categories or classes. It's more challenging when the number of text documents to be classified increases to several hundred thousands or millions. This is where techniques like feature extraction and supervised or unsupervised ML come in handy. Document classification is a generic problem not limited to text alone but also can be extended for other items like music, images, video, and other media.

To formalize our problem more clearly, we will have a given set of classes or categories and several text documents. Remember that documents are basically sentences or paragraphs of text. This forms a corpus. Our task would be to determine which class or classes each document belongs to. This entire process involves several steps which we will be discussing in detail later in this chapter. Briefly, for a supervised classification problem, we need to have some labelled data that we could use for training a text classification model. This data would essentially be curated documents that are already assigned to some specific class or category beforehand. Using this, we would essentially

extract features and attributes from each document and make our model learn these attributes corresponding to each particular document and its class/category by feeding it to a supervised ML algorithm. Of course, the data would need to be pre-processed and normalized before building the model. Once done, we would follow the same process of normalization and feature extraction and then feed it to the model to predict the class or category for new documents. However, for an unsupervised classification problem, we would essentially not have any pre-labelled training documents. We would use techniques like clustering and document similarity measures to cluster documents together based on their inherent properties and assign labels to them.

In this chapter, we will discuss the concept of text classification and how it can be formulated as a supervised ML problem. We will also talk about the various forms of classification and what they indicate. A clear depiction for the essential steps necessary to complete a text classification workflow will also be presented, and we will be covering some of the essential steps from the same workflow, which have not been discussed before, including feature extraction, classifiers, model evaluation, and finally we will put them all together in building a text classification system on real-world data.

What Is Text Classification?

Before we define text classification, we need to understand the scope of textual data and what we really mean by *classification*. The textual data involved here can be anything ranging from a phrase, sentence, or a complete document with paragraphs of text, which can be obtained from corpora, blogs, or anywhere from the Web. Text classification is also often called *document classification* just to cover all forms of textual content under the word *document*. The word *document* could be defined as some form of concrete representation of thoughts or events that could be in the form of writing, recorded speech, drawings, or presentations. I use the term *document* here to represent textual data such as sentences or paragraphs belonging to the English language.

Text classification is also often called *text categorization*, although I explicitly use the word *classification* here for two reasons. First, it depicts the same essence as text categorization, where we want to classify documents. The second reason is to also show that we would be using classification or a supervised ML approach here to classify or categorize the text. Text categorization can be done in many ways, as mentioned. We will be focusing explicitly on a supervised approach using classification. The process of classification is not restricted to text alone. It is used quite frequently in other domains including science, healthcare, weather forecasting, and technology.

Text or document classification is the process of assigning text documents into one or more classes or categories, assuming that we have a predefined set of classes. Documents here are textual documents, and each document can contain a sentence or even a paragraph of words. A text classification system would successfully be able to classify each document to its correct class(es) based on inherent properties of the document. Mathematically, we can define it like this: given some description and attributes d for a document D , where $d \in D$, and we have a set of predefined classes or

categories, $C = \{c_1, c_2, c_3, \dots, c_n\}$. The actual document D can have many inherent

properties and attributes that lead it to being an entity in a high-dimensional space. Using a subset of that space with a limit set of descriptions and features depicted by d , we

should be able to successfully assign the original document D to its correct class C_x using a text classification system T . This can be represented by $T : D \rightarrow C_x$.

We will talk more about the text classification system in detail later in the chapter. Figure 4-1 shows a high-level conceptual representation of the text classification process.

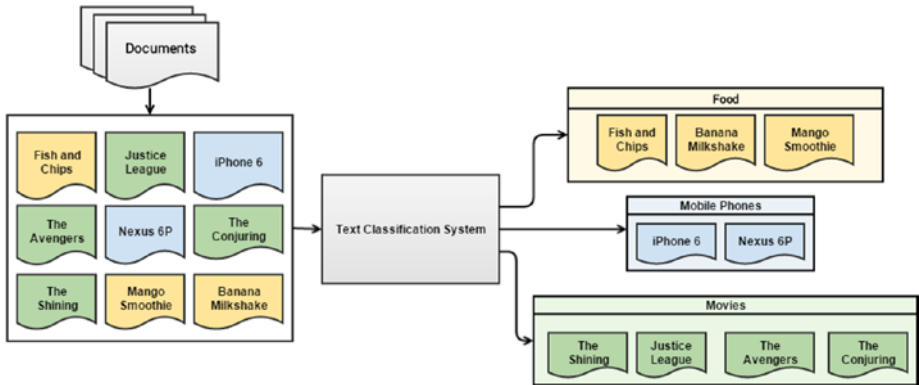


Figure 4-1. Conceptual overview of text classification

In Figure 4-1, we can see there are several documents representing products which can be assigned to various categories of food, mobile phones, and movies. Initially, these documents are all present together, just as a text corpus has various documents in it. Once it goes through a text classification system, represented as a black box here, we can see that each document is assigned to one specific class or category we had defined previously. Here the documents are just represented by their names, but in real data, they can contain much more, including descriptions of each product, specific attributes such as movie genre, product specifications, constituents, and many more properties that can be used as features in the text classification system to make document identification and classification easier.

There are various types of text classification. This chapter focuses on two major types, which are based on the type of content that makes up the documents:

- Content-based classification
- Request-based classification

Both types are more like different philosophies or ideals behind approaches to classifying text documents rather than specific technical algorithms or processes. *Content-based classification* is the type of text classification where priorities or weights are given to specific subjects or topics in the text content that would help determine the class of the document. A conceptual example would be that a book with more than 30 percent of its content about food preparations can be classified under cooking/recipes. *Request-based classification* is influenced by user requests and is targeted towards specific user groups and audiences. This type of classification is governed by specific policies and ideals.

Automated Text Classification

We now have an idea of the definition and scope of text classification. We have also formally defined text classification both conceptually and mathematically, where we talked about a “text classification system” being able to classify text documents to their respective categories or classes. Consider several humans doing the task of going through each document and classifying it. They would then be a part of the text classification system we are talking about. However, that would not scale very well once there were millions of text documents to be classified quickly. To make the process more efficient and faster, we can consider automating the task of text classification, which brings us to *automated text classification*.

To automate text classification, we can make use of several ML techniques and concepts. There are mainly two types of ML techniques that are relevant to solving this problem:

- Supervised machine learning
- Unsupervised machine learning

Besides these two techniques, there are also other families of learning algorithms, such as *reinforcement learning* and *semi-supervised learning*. Let us look at both supervised and unsupervised learning algorithms in more detail, from both an ML perspective how it can be leveraged in classifying text documents.

Unsupervised learning refers to specific ML techniques or algorithms that do not require any pre-labelled training data samples to build a model. We usually have a collection of data points, which could be text or numeric, depending on the problem we are trying to solve. We extract features from each of the data points using a process known as *feature extraction* and then feed the feature set for each data point into our algorithm. We are trying to extract meaningful patterns from the data, such as trying to group together similar data points using techniques like clustering or summarizing documents based on topic models. This is extremely useful in text document categorization and is also called document clustering, where we cluster documents into groups purely based on their features, similarity, and attributes, without training any model on previously labelled data. Later chapters further discuss unsupervised learning, covering topic models, document summarization, similarity analysis, and clustering.

Supervised learning refers to specific ML techniques or algorithms that are trained on pre-labelled data samples known as *training data*. Features or attributes are extracted from this data using feature extraction, and for each data point we will have its own feature set and corresponding class/label. The algorithm learns various patterns for each type of class from the training data. Once this process is complete, we have a *trained model*. This model can then be used to predict the class for future test data samples once we feed their features to the model. Thus the machine has actually learned, based on previous training data samples, how to predict the class for new unseen data samples.

There are two major types of supervised learning algorithms:

- *Classification*: The process of supervised learning is referred to as *classification* when the outcomes to be predicted are distinct categories, thus the outcome variable is a categorical variable in this case. Examples would be news categories or movie genres.

- *Regression*: Supervised learning algorithms are known as *regression* algorithms when the outcome we want to predict is a continuous numeric variable. Examples would be house prices or people's weights.

We will be specifically focusing on classification for our problem (hence the name of the chapter—we are trying to classify or categorize text documents into distinct classes or categories). We will be following a supervised learning approach in our implementations later on.

Now we are ready to define the process of automated or ML-based text classification mathematically. Say we have a training set of documents labelled with their corresponding class or category. This can be represented by TS , which is a set of paired documents and labels, $TS = \{(d_1, c_1), (d_2, c_2), \dots, (d_n, c_n)\}$ where $d_1, d_2, \dots, d_n$ is the list of text documents,

and their corresponding labels are $c_1, c_2, \dots, c_n$ such that $c_x \in C = \{c_1, c_2, \dots, c_n\}$ where c_x

denotes the class label for document x and C denotes the set of all possible distinct classes, any of which can be the class or classes for each document. Assuming we have our training set, we can define a supervised learning algorithm F such that when it is trained on our training dataset TS , we build a classification model or classifier γ such that we can say that $F(TS) = \gamma$. Thus the supervised learning algorithm F takes the input set of (*document*, *class*) pairs TS and gives us the trained classifier γ , which is our model. This process is known as the *training process*.

This model can then take a new, previously unseen document ND and predict its class c_{ND} such that $c_{ND} \in C$. This process is known as the *prediction process* and can be represented by $\gamma : TD \rightarrow c_{ND}$. Thus we can see that there are two main processes in the supervised text classification process:

- Training
- Prediction

An important point to remember is that some manually labelled training data is necessary for supervised text classification, so even though we are talking about automated text classification, to kick start the process we need some manual efforts. Of course, the benefits of this are manifold because once we have a trained classifier, we can keep using it to predict and classify new documents with minimal efforts and manual supervision.

There are various learning methods or algorithms that we will be discussing in a future section. These learning algorithms are not specific to text data but are generic ML algorithms that can be applied toward various types of data after due pre-processing and feature extraction. I will touch upon a couple of supervised ML algorithms and use them in solving a real-world text classification problem. These algorithms are usually trained on the training data set and often an optional validation set such that the model that is trained does not overfit to the training data, which basically means it would then not be able to generalize well and predict properly for new instances of text documents. Often the model is tuned on several of its internal parameters based on the learning algorithm and by evaluating various performance metrics like accuracy on the validation set or by using cross-validation where we split the training dataset itself into training and

validation sets by random sampling. This comprises the training process, the outcome of which yields a fully trained model that is ready to predict. In the prediction stage, we usually have new data points from the test dataset. We can use them to feed into the model after normalization and feature extraction and see how well the model is performing by evaluating its prediction performance.

There are a few types of text classification based on the number of classes to predict and the nature of predictions. These types of classification are based on the dataset, the number of classes/categories pertaining to that dataset, and the number of classes that can be predicted on any data point:

- *Binary classification* is when the total number of distinct classes or categories is two in number and any prediction can contain either one of those classes.
- *Multi-class classification*, also known as *multinomial classification*, refers to a problem where the total number of classes is more than two, and each prediction gives one class or category that can belong to any of those classes. This is an extension of the binary classification problem where the total number of classes is more than two.
- *Multi-label classification* refers to problems where each prediction can yield more than one outcome/predicted class for any data point.

Text Classification Blueprint

Now that we know the basic scope of automated text classification, this section will look at a blueprint for a complete workflow of building an automated text classifier system. This will consist of a series of steps that must be followed in both the training and testing phases mentioned in the earlier section. For building a text classification system, we need to make sure we have our source of data and retrieve that data so that we can start feeding it to our system. The following main steps outline a typical workflow for a text classification system, assuming we have our dataset already downloaded and ready to be used:

1. Prepare train and test datasets
2. Text normalization
3. Feature extraction
4. Model training
5. Model prediction and evaluation
6. Model deployment

These steps are carried out in that order for building a text classifier. Figure 4-2 shows a detailed workflow for a text classification system with the main processes highlighted in training and prediction.

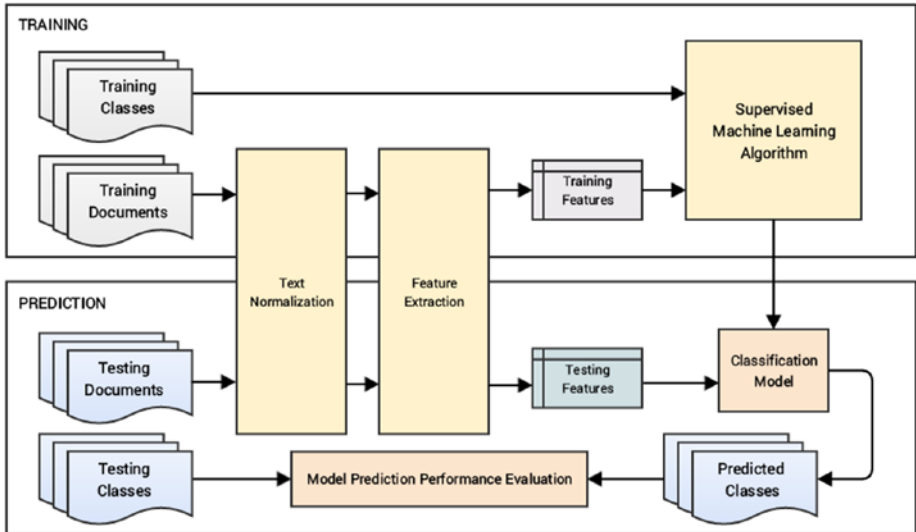


Figure 4-2. Blueprint for building an automated text classification system

Notice that there are two main boxes for Training and Prediction, which are the two main processes involved in building a text classifier. In general, the dataset we have is usually divided into two or three splits called the *training*, *validation* (optional), and *testing datasets*, respectively. You can see an overlap of the Text Normalization and Feature Extraction modules in Figure 4-2 for both processes, indicating that no matter which document we want to classify and predict its class, it must go through the same series of transformations in both the training and prediction process. Each document is first pre-processed and normalized, and then specific features pertaining to the document are extracted. These processes are always uniform in both the training and prediction processes to make sure that our classification model performs consistently in its predictions.

In the Training process, each document has its own corresponding class/category that was manually labeled or curated beforehand. These training text documents are processed and normalized in the Text Normalization module, giving us clean and standardized training text documents. They are then passed to the Feature Extraction module where different types of feature-extraction techniques are used to extract meaningful features from the processed text documents. We will cover some popular feature extraction techniques in a future section. These features are usually numeric arrays or vectors because standard ML algorithms work on numeric vectors. Once we have our features, we select a supervised ML algorithm and train our model.

Training the model involves feeding the feature vectors for the documents and the corresponding labels such that the algorithm is able to learn various patterns corresponding to each class/category and can reuse this learned knowledge to predict classes for future new documents. Often an optional validation dataset is used to evaluate the performance of the classification algorithm to make sure it generalizes well with the data during training. A combination of these features and the ML algorithm yields a Classification Model, which is the end stage of the Training process. Often this model is tuned using various model parameters with a process called *hyperparameter tuning* to build a better performing optimal model.

The Prediction process shown in the figure involves trying to either predict classes for new documents or evaluating how predictions are working on testing data. The test dataset documents go through the same process of normalization and feature extraction, and the test document features are passed to the trained Classification Model, which predicts the possible class for each of the documents based on previously learned patterns. If you have the true class labels for the documents that were manually labelled, you can evaluate the prediction performance of the model by comparing the true labels and the predicted labels using various metrics like accuracy. This would give an idea of how well the model performs its predictions for new documents.

Once we have a stable and working model, the last step is usually deploying the model, which normally involves saving the model and its necessary dependencies and deploying it as a service or as a running program that predicts categories for new documents as a batch job, or based on serving user requests if accessed as a web service. There are various ways to deploy ML models, and this usually depends on how you want to access it later on.

We will now discuss some of the main modules from the preceding blueprint and implement these modules so that we can integrate them all together to build a real-world text classifier.

Text Normalization

Chapter 3 covered text processing and normalization in detail—refer it to see the various methods and techniques available. In this section, we will define a normalizer module to normalize text documents and will be using it later when we build our classifier. Although various techniques are available, we will keep it fairly simple and straightforward here so that it is not too hard to follow our implementations step by step. We will implement and use the following normalization techniques in our module:

- Expanding contractions
- Text standardization through lemmatization
- Removing special characters and symbols
- Removing stopwords

We are not focusing too much on correcting spellings and other advanced techniques, but you can integrate the functions from the previous chapter implementation if you are interested. Our normalization module is implemented and available in `normalization.py`, available in the code files for this chapter. I will also be explaining each function here for your convenience. We will first start with loading the necessary dependencies. Remember that you will need our custom-defined contractions mapping file from Chapter 3, named `contractions.py`, for expanding contractions.

The following snippet shows the necessary imports and dependencies:

```
from contractions import CONTRACTION_MAP
import re
import nltk
import string
from nltk.stem import WordNetLemmatizer
```

```
stopword_list = nltk.corpus.stopwords.words('english')
wnl = WordNetLemmatizer()
```

We load all the English stopwords, the contraction mappings in `CONTRACTION_MAP`, and an instance of `WordNetLemmatizer` for carrying our lemmatization. We now define a function to tokenize text into tokens that will be used by our other normalization functions. The following function tokenizes and removes any extraneous whitespace from the tokens:

```
def tokenize_text(text):
    tokens = nltk.word_tokenize(text)
    tokens = [token.strip() for token in tokens]
    return tokens
```

Now we define a function for expanding contractions. This function is similar to our implementation from Chapter 3—it takes in a body of text and returns the same with its contractions expanded if there is a match. The following snippet helps us achieve this:

```
def expand_contractions(text, contraction_mapping):

    contractions_pattern = re.compile('{}'.format('|'.join(contraction_
mapping.keys()))),
                                flags=re.IGNORECASE|re.DOTALL)

    def expand_match(contraction):
        match = contraction.group(0)
        first_char = match[0]
        expanded_contraction = contraction_mapping.get(match)\
            if contraction_mapping.get(match)\
            else contraction_mapping.get(match.lower())
        expanded_contraction = first_char+expanded_contraction[1:]
        return expanded_contraction

    expanded_text = contractions_pattern.sub(expand_match, text)
    expanded_text = re.sub("'", "", expanded_text)
    return expanded_text
```

Now that we have a function for expanding contractions, we implement a function for standardizing our text data by bringing word tokens to their base or root form using lemmatization. The following functions will help us in achieving that:

```
from pattern.en import tag
from nltk.corpus import wordnet as wn

# Annotate text tokens with POS tags
def pos_tag_text(text):
    # convert Penn treebank tag to wordnet tag
```

```

def penn_to_wn_tags(pos_tag):
    if pos_tag.startswith('J'):
        return wn.ADJ
    elif pos_tag.startswith('V'):
        return wn.VERB
    elif pos_tag.startswith('N'):
        return wn.NOUN
    elif pos_tag.startswith('R'):
        return wn.ADV
    else:
        return None

tagged_text = tag(text)
tagged_lower_text = [(word.lower(), penn_to_wn_tags(pos_tag))
                     for word, pos_tag in
                     tagged_text]
return tagged_lower_text

# lemmatize text based on POS tags
def lemmatize_text(text):

    pos_tagged_text = pos_tag_text(text)
    lemmatized_tokens = [wnl.lemmatize(word, pos_tag) if pos_tag
                        else word
                        for word, pos_tag in pos_tagged_text]
    lemmatized_text = ' '.join(lemmatized_tokens)
    return lemmatized_text

```

The preceding snippet depicts two functions implemented for lemmatization. The main function is `lemmatize_text`, which takes in a body of text data and lemmatizes each word of the text based on its POS tag if it is present and then returns the lemmatized text back to the user. For this, we need to annotate the text tokens with their POS tags. We use the `tag` function from `pattern` to annotate POS tags for each token and then further convert the POS tags from the Penn treebank syntax to WordNet syntax, since the `WordNetLemmatizer` checks for POS tag annotations based on WordNet formats. We convert each word token to lowercase, annotate it with its correct, converted WordNet POS tag, and return these annotated tokens, which are finally fed into the `lemmatize_text` function.

The following function helps us remove special symbols and characters:

```

def remove_special_characters(text):
    tokens = tokenize_text(text)
    pattern = re.compile('[{}]' .format(re.escape(string.punctuation)))
    filtered_tokens = filter(None, [pattern.sub('', token) for token in
    tokens])
    filtered_text = ' '.join(filtered_tokens)
    return filtered_text

```

We remove special characters by tokenizing the text just so we can remove some of the tokens that are actually contractions, but we may have failed to remove them in our first step, like "s" or "re". We will do this when we remove stopwords. However, you can also remove special characters without tokenizing the text. We remove all special symbols defined in `string.punctuation` from our text using regular expression matches. The following function helps us remove stopwords from our text data:

```
def remove_stopwords(text):
    tokens = tokenize_text(text)
    filtered_tokens = [token for token in tokens if token not in
                       stopwords_list]
    filtered_text = ' '.join(filtered_tokens)
    return filtered_text
```

Now that we have all our functions defined, we can build our text normalization pipeline by chaining all these functions one after another. The following function implements this, where it takes in a corpus of text documents and normalizes them and returns a normalized corpus of text documents:

```
def normalize_corpus(corpus, tokenize=False):

    normalized_corpus = []
    for text in corpus:
        text = expand_contractions(text, CONTRACTION_MAP)
        text = lemmatize_text(text)
        text = remove_special_characters(text)
        text = remove_stopwords(text)
        normalized_corpus.append(text)
        if tokenize:
            text = tokenize_text(text)
            normalized_corpus.append(text)

    return normalized_corpus
```

That brings us to the end of our discussion and implementation of necessary functions for our text normalization module. We will now look at concepts and practical implementation for feature extraction.

Feature Extraction

There are various feature-extraction techniques that can be applied on text data, but before we jump into them, let us consider what we mean by features. Why do we need them, and how they are useful? In a dataset, there are typically many data points. Usually the rows of the dataset and the columns are various features or properties of the dataset, with specific values for each row or observation. In ML terminology, *features* are unique, measurable attributes or properties for each observation or data point in a dataset. Features are usually numeric in nature and can be absolute numeric values or categorical

features that can be encoded as binary features for each category in the list using a process called *one-hot encoding*. The process of extracting and selecting features is both art and science, and this process is called *feature extraction* or *feature engineering*.

Usually extracted features are fed into ML algorithms for learning patterns that can be applied on future new data points for getting insights. These algorithms usually expect features in the form of numeric vectors because each algorithm is at heart a mathematical operation of optimization and minimizing loss and error when it tries to learn patterns from data points and observations. So, with textual data there is the added challenge of figuring out how to transform textual data and extract numeric features from it.

Now we will look at some feature-extraction concepts and techniques specially aligned towards text data.

The *Vector Space Model* is a concept and model that is very useful in case we are dealing with textual data and is very popular in information retrieval and document ranking. The Vector Space Model, also known as the *Term Vector Model*, is defined as a mathematical and algebraic model for transforming and representing text documents as numeric vectors of specific terms that form the vector dimensions. Mathematically this can be defined as follows. Say we have a document D in a document vector space VS . The number of dimensions or columns for each document will be the total number of distinct terms or words for all documents in the vector space. So, the vector space can be denoted

$$VS = \{W_1, W_2, \dots, W_n\}$$

where there are n distinct words across all documents. Now we can represent document D in this vector space as

$$D = \{w_{D1}, w_{D2}, \dots, w_{Dn}\}$$

where w_{Dn} denotes the weight for word n in document D . This weight is a numeric value and can represent anything, ranging from the frequency of that word in the document, to the average frequency of occurrence, or even to the TF-IDF weight (discussed shortly).

We will be talking about and implementing the following feature-extraction techniques:

- Bag of Words model
- TF-IDF model
- Advanced word vectorization models

An important thing to remember for feature extraction is that once we build a feature extractor using some transformations and mathematical operations, we need to make sure we reuse the same process when extracting features from new documents to be predicted, and not rebuild the whole algorithm again based on the new documents. We will be depicting this also with an example for each technique. Do note that for implementations based on practical examples in this section, we will be making use of the `nlTK`, `gensim`, and `scikit-learn` libraries, which you can install using `pip` as discussed earlier (in case you do not have them installed already).

The implementations are divided into two major modules. The file `feature_extractors.py` contains the generic functions we will be using later on when building the classifier, and we have used the same functions in the `feature_extraction_demo.py` file to show how each technique works with some practical examples. You can access them from the code files, and as always I will be presenting the same code in this chapter for ease of understanding. We will be using the following documents depicted in the `CORPUS` variable to extract features from and building some of the vectorization models. To illustrate how feature extraction will work for a new document (as a part of test dataset), we will also use a separate document as shown in the variable `new_doc` in the following snippet:

```
CORPUS = [
    'the sky is blue',
    'sky is blue and sky is beautiful',
    'the beautiful sky is so blue',
    'i love blue cheese'
]

new_doc = ['loving this blue sky today']
```

Bag of Words Model

The Bag of Words model is perhaps one of the simplest yet most powerful techniques to extract features from text documents. The essence of this model is to convert text documents into vectors such that each document is converted into a vector that represents the frequency of all the distinct words that are present in the document vector space for that specific document. Thus, considering our sample vector from the previous mathematical notation for D , the weight for each word is equal to its frequency of occurrence in that document.

An interesting thing is that we can even create the same model for individual word occurrences as well as occurrences for n -grams, which would make it an n -gram Bag of Words model such that frequency of distinct n -grams in each document would also be considered.

The following code snippet gives us a function that implements a Bag of Words-based feature-extraction model that also accepts an `ngram_range` parameter to take into account n -grams as features:

```
from sklearn.feature_extraction.text import CountVectorizer

def bow_extractor(corpus, ngram_range=(1,1)):

    vectorizer = CountVectorizer(min_df=1, ngram_range=ngram_range)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features
```

The preceding function uses the `CountVectorizer` class. You can access its detailed API (Application Programming Interface) documentation at http://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.

`html#sklearn.feature_extraction.text.CountVectorizer`, which has a whole bunch of various parameters for more fine-tuning based on the type of features you want to extract. We use its default configuration, which is enough for most scenarios, with `min_df` set to 1 indicating taking terms having a minimum frequency of 1 in the overall document vector space. You can set `ngram_range` to various parameters like (1, 3) would build feature vectors consisting of all unigrams, bigrams, and trigrams. The following snippet shows the function in action on our sample corpora of four training documents and one test document:

```
# build bow vectorizer and get features
In [371]: bow_vectorizer, bow_features = bow_extractor(CORPUS)
...: features = bow_features.todense()
...: print features
[[0 0 1 0 1 0 1 0 1]
 [1 1 1 0 2 0 2 0 0]
 [0 1 1 0 1 0 1 1 1]
 [0 0 1 1 0 1 0 0 0]]

# extract features from new document using built vectorizer
In [373]: new_doc_features = bow_vectorizer.transform(new_doc)
...: new_doc_features = new_doc_features.todense()
...: print new_doc_features
[[0 0 1 0 0 0 1 0 0]]

# print the feature names
In [374]: feature_names = bow_vectorizer.get_feature_names()
...: print feature_names
[u'and', u'beautiful', u'blue', u'cheese', u'is', u'love', u'sky', u'so',
 u'the']
```

That output shows how each text document has been converted to vectors. Each row represents one document from our corpus, and we do the same for both our corpora. The vectorizer is built using documents from CORPUS. We extract features from it and also use this built vectorizer to extract features from a completely new document. Each column in a vector represents the words depicted in `feature_names`, and the value is the frequency of that word in the document represented by the vector. It may be hard to comprehend this at first glance, so I have prepared the following function, which I hope you can use to understand the feature vectors better:

```
import pandas as pd

def display_features(features, feature_names):
    df = pd.DataFrame(data=features,
                      columns=feature_names)
    print df
```

Now you can feed the feature names and vectors to this function and see the feature matrix in a much easier-to-understand structure, shown here:

```
In [379]: display_features(features, feature_names)
```

	and	beautiful	blue	cheese	is	love	sky	so	the
0	0		0	1	0	1	0	1	0
1	1		1	1	0	2	0	2	0
2	0		1	1	0	1	0	1	1
3	0		0	1	1	0	1	0	0

```
In [380]: display_features(new_doc_features, feature_names)
```

	and	beautiful	blue	cheese	is	love	sky	so	the
0	0		0	1	0	0	0	1	0

That makes things much clearer, right? Consider the second document of CORPUS, represented in the preceding in row 1 of the first table. You can see that 'sky is blue and sky is beautiful' has value 2 for the feature sky, 1 for beautiful, and so on. Values of 0 are assigned for words not present in the document. Note that for the new document `new_doc`, there is no feature for the words *today*, *this*, or *loving* in the sentence. The reason for this is what I mentioned before—that the feature-extraction process, model, and vocabulary are always based on the training data and will never change or get influenced on newer documents, which it will predict later as a part of testing or otherwise. You might have guessed that this is because a model is always trained on some training data and is never influenced by newer documents unless we plan on rebuilding that model. Hence, the features in this model are always limited based on the document vector space of the training corpus.

You have now started to get an idea of how to extract meaningful vector-based features from text data, which previously seemed impossible. Try out the preceding functions by setting `ngram_range` to (1, 3) and see the outputs.

TF-IDF Model

The Bag of Words model is good, but the vectors are completely based on absolute frequencies of word occurrences. This has some potential problems where words that may tend to occur a lot across all documents in the corpus will have higher frequencies and will tend to overshadow other words that may not occur as frequently but may be more interesting and effective as features to identify specific categories for the documents. This is where TF-IDF comes into the picture. TF-IDF stands for Term Frequency-Inverse Document Frequency, a combination of two metrics: *term frequency* and *inverse document frequency*. This technique was originally developed as a metric for ranking functions for showing search engine results based on user queries and has come to be a part of information retrieval and text feature extraction now.

Let us formally define TF-IDF now and look at the mathematical representations for it before diving into its implementation. Mathematically, TF-IDF is the product of two metrics and can be represented as $tfidf = tf \times idf$, where term frequency (*tf*) and inverse-document frequency (*idf*) represent the two metrics.

Term frequency denoted by *tf* is what we had computed in the Bag of Words model. Term frequency in any document vector is denoted by the raw frequency value of that term in a particular document. Mathematically it can be represented as $tf(w, D) = f_{w_D}$, where f_{w_D} denotes frequency for word *w* in document *D*, which becomes the term

frequency (*tf*). There are various other representations and computations for term frequency, such as converting frequency to a binary feature where 1 means the term has occurred in the document and 0 means it has not. Sometimes you can also normalize the absolute raw frequency using logarithms or averaging the frequency. We will be using the raw frequency in our computations.

Inverse document frequency denoted by *idf* is the inverse of the document frequency for each term. It is computed by dividing the total number of documents in our corpus by the document frequency for each term and then applying logarithmic scaling on the result. In our implementation we will be adding 1 to the document frequency for each term just to indicate that we also have one more document in our corpus that essentially has every term in the vocabulary. This is to prevent potential division-by-zero errors and smoothen the inverse document frequencies. We also add 1 to the result of our *idf* computation to avoid ignoring terms completely that might have zero *idf*. Mathematically our implementation for *idf* can be represented by

$$idf(t) = 1 + \log \frac{C}{1 + df(t)}$$

where *idf*(*t*) represents the *idf* for the term *t*, *C* represents the count of the total number of documents in our corpus, and *df*(*t*) represents the frequency of the number of documents in which the term *t* is present.

Thus the term frequency-inverse document frequency can be computed by multiplying the above two measures together. The final TF-IDF metric we will be using is a normalized version of the *tfidf* matrix we get from the product of *tf* and *idf*. We will normalize the *tfidf* matrix by dividing it with the L2 norm of the matrix, also known as the *Euclidean norm*, which is the square root of the sum of the square of each term's *tfidf* weight. Mathematically we can represent the final *tfidf* feature vector as $tfidf = \frac{tfidf}{\|tfidf\|}$,

where $\|tfidf\|$ represents the Euclidean L2 norm for the *tfidf* matrix.

The following code snippet shows an implementation of getting the *tfidf*-based feature vectors, considering we have our Bag of Words feature vectors we obtained in the previous section:

```
from sklearn.feature_extraction.text import TfidfTransformer

def tfidf_transformer(bow_matrix):

    transformer = TfidfTransformer(norm='l2',
                                    smooth_idf=True,
                                    use_idf=True)
    tfidf_matrix = transformer.fit_transform(bow_matrix)
    return transformer, tfidf_matrix
```

You can see that we have used the L2 norm option in the parameters and also made sure we smoothen the *idfs* to give weightages also to terms that may have zero *idf* so that we do not ignore them. We can see this function in action in the following code snippet:

```

import numpy as np
from feature_extractors import tfidf_transformer
feature_names = bow_vectorizer.get_feature_names()

# build tfidf transformer and show train corpus tfidf features
In [388]: tfidf_trans, tfidf_features = tfidf_transformer(bow_features)
...: features = np.round(tfidf_features.todense(), 2)
...: display_features(features, feature_names)
    and beautiful blue cheese is love sky so the
0 0.00      0.00 0.40    0.00 0.49 0.00 0.49 0.00 0.60
1 0.44      0.35 0.23    0.00 0.56 0.00 0.56 0.00 0.00
2 0.00      0.43 0.29    0.00 0.35 0.00 0.35 0.55 0.43
3 0.00      0.00 0.35    0.66 0.00 0.66 0.00 0.00 0.00

# show tfidf features for new_doc using built tfidf transformer
In [389]: nd_tfidf = tfidf_trans.transform(new_doc_features)
...: nd_features = np.round(nd_tfidf.todense(), 2)
...: display_features(nd_features, feature_names)
    and beautiful blue cheese is love sky so the
0 0.0      0.0 0.63    0.0 0.0 0.0 0.77 0.0 0.0

```

Thus the preceding outputs show the *tfidf* feature vectors for all our sample documents. We use the `TfidfTransformer` class, which helps us in computing the *tfidfs* for each document based on the equations described earlier.

Now we will show how the internals of this class work. You will also see how to implement the mathematical equations described earlier to compute the *tfidf*-based feature vectors. This section is dedicated to ML experts (and curious readers who are interested in how things work behind the scenes). We will start with loading necessary dependencies and computing the term frequencies (*TF*) by reusing our Bag of Words-based features for our sample corpus, which can also act as the term frequencies for our training CORPUS:

```

import scipy.sparse as sp
from numpy.linalg import norm
feature_names = bow_vectorizer.get_feature_names()

# compute term frequency
tf = bow_features.todense()
tf = np.array(tf, dtype='float64')

# show term frequencies
In [391]: display_features(tf, feature_names)
    and beautiful blue cheese is love sky so the
0 0.0      0.0 1.0    0.0 1.0 0.0 1.0 0.0 1.0
1 1.0      1.0 1.0    0.0 2.0 0.0 2.0 0.0 0.0
2 0.0      1.0 1.0    0.0 1.0 0.0 1.0 1.0 1.0
3 0.0      0.0 1.0    1.0 0.0 1.0 0.0 0.0 0.0

```

We will now compute our document frequencies (*DF*) for each term based on the number of documents in which it occurs. The following snippet shows how to obtain it from our Bag of Words feature matrix:

```
# build the document frequency matrix
df = np.diff(sp.csc_matrix(bow_features, copy=True).indptr)
df = 1 + df # to smoothen idf later

# show document frequencies
In [403]: display_features([df], feature_names)
and beautiful blue cheese is love sky so the
0 2 3 5 2 4 2 4 2 3
```

This tells us the document frequency (*DF*) for each term and you can verify it with the documents in CORPUS. Remember that we have added 1 to each frequency value to smoothen the *idf* values later and prevent division-by-zero errors by assuming we have a document (imaginary) that has all the terms once. Thus, if you check in the CORPUS, you will see that blue occurs 4(+1) times, sky occurs 3(+1) times, and so on, considering (+1) for our smoothening.

Now that we have the document frequencies, we will compute the inverse document frequency (*idf*) using our formula defined earlier. Remember to add 1 to the total count of documents in the corpus to add the document that we had assumed earlier to contain all the terms at least once for smoothening the *idfs*:

```
# compute inverse document frequencies
total_docs = 1 + len(CORPUS)
idf = 1.0 + np.log(float(total_docs) / df)

# show inverse document frequencies
In [406]: display_features([np.round(idf, 2)], feature_names)
and beautiful blue cheese is love sky so the
0 1.92 1.51 1.0 1.92 1.22 1.92 1.22 1.92 1.51
```

```
# compute idf diagonal matrix
total_features = bow_features.shape[1]
idf_diag = sp.spdiags(idf, diags=0, m=total_features, n=total_features)
idf = idf_diag.todense()
```

```
# print the idf diagonal matrix
In [407]: print np.round(idf, 2)
[[ 1.92  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  1.51 0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  1.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  1.92 0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  1.22 0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  1.92 0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  1.22 0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  1.92 0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  1.51]]
```

You can now see the *idf* matrix that we created based on our mathematical equation, and we also convert it to a diagonal matrix, which will be helpful later on when we want to compute the product with term frequency.

Now that we have our *tfs* and *idfs*, we can compute the *tfidf* feature matrix using matrix multiplication, as shown in the following snippet:

```
# compute tfidf feature matrix
tfidf = tf * idf

# show tfidf feature matrix
In [410]: display_features(np.round(tfidf, 2), feature_names)
      and beautiful blue cheese is love sky so the
0  0.00      0.00  1.0   0.00  1.22  0.00  1.22  0.00  1.51
1  1.92      1.51  1.0   0.00  2.45  0.00  2.45  0.00  0.00
2  0.00      1.51  1.0   0.00  1.22  0.00  1.22  1.92  1.51
3  0.00      0.00  1.0   1.92  0.00  1.92  0.00  0.00  0.00
```

We now have our *tfidf* feature matrix, but wait! It is not yet over. We have to divide it with the L2 norm, if you remember from our equations depicted earlier. The following snippet computes the *tfidf* norms for each document and then divides the *tfidf* weights with the norm to give us the final desired *tfidf* matrix:

```
# compute L2 norms
norms = norm(tfidf, axis=1)

# print norms for each document
In [412]: print np.round(norms, 2)
[ 2.5  4.35  3.5  2.89]

# compute normalized tfidf
norm_tfidf = tfidf / norms[:, None]

# show final tfidf feature matrix
In [415]: display_features(np.round(norm_tfidf, 2), feature_names)
      and beautiful blue cheese is love sky so the
0  0.00      0.00  0.40   0.00  0.49  0.00  0.49  0.00  0.60
1  0.44      0.35  0.23   0.00  0.56  0.00  0.56  0.00  0.00
2  0.00      0.43  0.29   0.00  0.35  0.00  0.35  0.55  0.43
3  0.00      0.00  0.35   0.66  0.00  0.66  0.00  0.00  0.00
```

Compare the preceding obtained *tfidf* feature matrix for the documents in CORPUS to the feature matrix obtained using *TfidfTransformer* earlier. Note they are exactly the same, thus verifying that our mathematical implementation was correct—and in fact this very same implementation is adopted by *scikit-learn*'s *TfidfTransformer* behind the scenes using some more optimizations. Now, suppose we want to compute the *tfidf*-based feature matrix for our new document *new_doc*. We can do it using the following snippet. We reuse the *new_doc_features* Bag of Words vector from before for the term frequencies:

```
# compute new doc term freqs from bow freqs
nd_tf = new_doc_features
nd_tf = np.array(nd_tf, dtype='float64')

# compute tfidf using idf matrix from train corpus
nd_tfidf = nd_tf*idf
nd_norms = norm(nd_tfidf, axis=1)
norm_nd_tfidf = nd_tfidf / nd_norms[:, None]

# show new_doc tfidf feature vector
In [418]: display_features(np.round(norm_nd_tfidf, 2), feature_names)
and beautiful blue cheese is love sky so the
0 0.0      0.0 0.63    0.0 0.0   0.0 0.77 0.0 0.0
```

The preceding output depicts the *tfidf*-based feature vector for `new_doc`, and you can see it is the same as the one obtained by `TfidfTransformer`.

Now that we know how the internals work, we are going to implement a generic function that can directly compute the *tfidf*-based feature vectors for documents from the raw documents themselves. The following snippet depicts the same:

```
from sklearn.feature_extraction.text import TfidfVectorizer

def tfidf_extractor(corpus, ngram_range=(1,1)):

    vectorizer = TfidfVectorizer(min_df=1,
                                norm='l2',
                                smooth_idf=True,
                                use_idf=True,
                                ngram_range=ngram_range)
    features = vectorizer.fit_transform(corpus)
    return vectorizer, features
```

The preceding function makes use of the `TfidfVectorizer`, which directly computes the *tfidf* vectors by taking the raw documents themselves as input and internally computing the term frequencies as well as the inverse document frequencies, eliminating the need to use the `CountVectorizer` for computing the term frequencies based on the Bag of Words model. Support is also present for adding n-grams to the feature vectors. We can see the function in action in the following snippet:

```
# build tfidf vectorizer and get training corpus feature vectors
In [425]: tfidf_vectorizer, tfidf_features = tfidf_extractor(CORPUS)
...: display_features(np.round(tfidf_features.todense(), 2), feature_
    names)
and beautiful blue cheese is love sky so the
0 0.00      0.00 0.40    0.00 0.49 0.00 0.49 0.00 0.60
1 0.44      0.35 0.23    0.00 0.56 0.00 0.56 0.00 0.00
2 0.00      0.43 0.29    0.00 0.35 0.00 0.35 0.55 0.43
3 0.00      0.00 0.35    0.66 0.00 0.66 0.00 0.00 0.00
```

```
# get tfidf feature vector for the new document
In [426]: nd_tfidf = tfidf_vectorizer.transform(new_doc)
...: display_features(np.round(nd_tfidf.todense(), 2), feature_names)
and beautiful blue cheese is love sky so the
0 0.0          0.0 0.63      0.0 0.0    0.0 0.77 0.0 0.0
```

You can see from the preceding outputs that the *tfidf* feature vectors match to the ones we obtained previously. This brings us to the end of our discussion on feature extraction using *tfidf*. Now we will look at some advanced word vectorization techniques.

Advanced Word Vectorization Models

There are various approaches to creating more advanced word vectorization models for extracting features from text data. Here we will discuss a couple of them that use Google's popular word2vec algorithm. The word2vec model, released in 2013 by Google, is a neural network-based implementation that learns distributed vector representations of words based on continuous Bag of Words and skip-gram-based architectures. The word2vec framework is much faster than other neural network-based implementations and does not require manual labels to create meaningful representations among words. You can find more details on Google's word2vec project at <https://code.google.com/archive/p/word2vec/>. You can even try out some of the implementations yourself if you are interested.

We will be using the *gensim* library in our implementation, which is Python implementation for word2vec that provides several high-level interfaces for easily building these models. The basic idea is to provide a corpus of documents as input and get feature vectors for them as output. Internally, it constructs a vocabulary based on the input text documents and learns vector representations for words based on various techniques mentioned earlier, and once this is complete, it builds a model that can be used to extract word vectors for each word in a document. Using various techniques like average weighting or *tfidf* weighting, we can compute the averaged vector representation of a document using its word vectors. You can get more details about the interface for *gensim*'s word2vec implementation at <http://radimrehurek.com/gensim/models/word2vec.html>.

We will be mainly focusing on the following parameters when we build our model from our sample training corpus:

- **size:** This parameter is used to set the size or dimension for the word vectors and can range from tens to thousands. You can try out various dimensions to see which gives the best result.
- **window:** This parameter is used to set the context or window size, which specifies the length of the window of words that should be considered for the algorithm to take into account as context when training.
- **min_count:** This parameter specifies the minimum word count needed across the corpus for the word to be considered in the vocabulary. This helps in removing very specific words that may not have much significance because they occur very rarely in the documents.

- **sample:** This parameter is used to downsample effects of occurrence of frequent words. Values between 0.01 and 0.0001 are usually ideal.

Once we build a model, we will define and implement two techniques of combining word vectors together in text documents based on certain weighing schemes. We will implement two techniques mentioned as follows.

- Averaged word vectors
- TF-IDF weighted word vectors

Let us start the feature-extraction process by building our word2vec model on our sample training corpus before going into further implementations. The following code snippet shows how:

```
import gensim
import nltk

# tokenize corpora
TOKENIZED_CORPUS = [nltk.word_tokenize(sentence)
                     for sentence in CORPUS]
tokenized_new_doc = [nltk.word_tokenize(sentence)
                     for sentence in new_doc]

# build the word2vec model on our training corpus
model = gensim.models.Word2Vec(TOKENIZED_CORPUS, size=10, window=10,
                               min_count=2, sample=1e-3)
```

As you can see, we have built the model using the parameters described earlier; you can play around with these and also look at other parameters from the documentation to change the architecture type, number of workers, and so on. Now that we have our model ready, we can start implementing our feature extraction techniques.

Averaged Word Vectors

The preceding model creates a vector representation for each word in the vocabulary. We can access them by just typing in the following code:

```
In [430]: print model['sky']
[ 0.01608407 -0.04819566  0.04227461 -0.03011346  0.0254148  0.01728328
  0.0155535  0.00774884 -0.02752112  0.01646519]

In [431]: print model['blue']
[-0.0472235  0.01662185 -0.01221706 -0.04724348 -0.04384995  0.00193343
 -0.03163504 -0.03423524  0.02661656  0.03033725]
```

Each word vector is of length 10 based on the size parameter specified earlier. But when we deal with sentences and text documents, they are of unequal length, and we must carry out some form of combining and aggregation operations to make sure the

number of dimensions of the final feature vectors are the same, regardless of the length of the text document, number of words, and so on. In this technique, we will use an average weighted word vectorization scheme, where for each text document we will extract all the tokens of the text document, and for each token in the document we will capture the subsequent word vector if present in the vocabulary. We will sum up all the word vectors and divide the result by the total number of words matched in the vocabulary to get a final resulting averaged word vector representation for the text document. This can be mathematically represented using the equation

$$AVV(D) = \frac{\sum_1^n wv(w)}{n}$$

where $AVV(D)$ is the averaged word vector representation for document D , containing words $w_1, w_2, \dots, w_n$, and $wv(w)$ is the word vector representation for the word w .

The following snippet shows the pseudocode for the algorithm just described:

```
model := the word2vec model we built
vocabulary := unique_words(model)
document := [words]
matched_word_count := 0
vector := []

for word in words:
    if word in vocabulary:
        vector := vector + model[word]
        matched_word_count := matched_word_count + 1

averaged_word_vector := vector / matched_word_count
```

That snippet shows the flow of operations in a better way that is easier to understand. We will now implement our algorithm in Python using the following code snippet:

```
import numpy as np

# define function to average word vectors for a text document
def average_word_vectors(words, model, vocabulary, num_features):

    feature_vector = np.zeros((num_features,), dtype="float64")
    nwords = 0.

    for word in words:
        if word in vocabulary:
            nwords = nwords + 1.
            feature_vector = np.add(feature_vector, model[word])
```

```

    if nwords:
        feature_vector = np.divide(feature_vector, nwords)

    return feature_vector

# generalize above function for a corpus of documents
def averaged_word_vectorizer(corpus, model, num_features):
    vocabulary = set(model.index2word)
    features = [average_word_vectors(tokenized_sentence, model, vocabulary,
                                     num_features)
                for tokenized_sentence in corpus]
    return np.array(features)

```

The `average_word_vectors()` function must seem familiar to you—it is the concrete implementation of our algorithm shown using our pseudocode earlier. We also create a generic function `averaged_word_vectorizer()` to perform averaging of word vectors for a corpus of documents. The following snippet shows our function in action on our sample corpora:

```

# get averaged word vectors for our training CORPUS
In [445]: avg_word_vec_features = averaged_word_vectorizer(corpus=TOKENIZED_
CORPUS,
...:                                                         model=model,
...:                                                         num_features=10)
...: print np.round(avg_word_vec_features, 3)
[[ 0.006 -0.01  0.015 -0.014  0.004 -0.006 -0.024 -0.007 -0.001  0.    ]
 [-0.008 -0.01  0.021 -0.019 -0.002 -0.002 -0.011  0.002  0.003 -0.001]
 [-0.003 -0.007  0.008 -0.02  -0.001 -0.004 -0.014 -0.015  0.002 -0.01 ]
 [-0.047  0.017 -0.012 -0.047 -0.044  0.002 -0.032 -0.034  0.027  0.03 ]]

# get averaged word vectors for our test new_doc
In [447]: nd_avg_word_vec_features = averaged_word_
vectorizer(corpus=tokenized_new_doc,
...:                                                         model=model,
...:                                                         num_
...:                                                         features=10)
...: print np.round(nd_avg_word_vec_features, 3)
[[-0.016 -0.016  0.015 -0.039 -0.009  0.01  -0.008 -0.013  0.    0.023]]

```

From the preceding outputs, you can see that we have uniformly sized averaged word vectors for each document in the corpus, and these feature vectors can be used later for classification by feeding it to the ML algorithms.

TF-IDF Weighted Averaged Word Vectors

Our previous vectorizer simply sums up all the word vectors pertaining to any document based on the words in the model vocabulary and calculates a simple average by dividing with the count of matched words. This section introduces a new and novel technique

of weighing each matched word vector with the word TF-IDF score and summing up all the word vectors for a document and dividing it by the sum of all the TF-IDF weights of the matched words in the document. This would basically give us a TF-IDF weighted averaged word vector for each document.

This can be mathematically represented using the equation

$$TWA(D) = \frac{\sum_{w=1}^n wv(w) \times tfidf(w)}{n}$$

where $TWA(D)$ is the TF-IDF weighted averaged word vector representation for document D , containing words $w_1, w_2, \dots, w_n$, where $wv(w)$ is the word vector representation and $tfidf(w)$ is the TF-IDF weight for the word w . The following snippet shows the pseudocode for this algorithm:

```
model := the word2vec model we built
vocabulary := unique_words(model)
document := [words]
tfidfs := [tfidf(word) for each word in words]
matched_word_wts := 0
vector := []

for word in words:
    if word in vocabulary:
        word_vector := model[word]
        weighted_word_vector := tfidfs[word] x word_vector
        vector := vector + weighted_word_vector
        matched_word_wts := matched_word_wts + tfidfs[word]

tfidf_wtd_avgd_word_vector := vector / matched_word_wts
```

That pseudocode gives structure to our algorithm and shows how to implement the algorithm from the mathematical formula we defined earlier.

The following code snippet implements this algorithm in Python so we can use it for feature extraction:

```
# define function to compute tfidf weighted averaged word vector for a document
def tfidf_wtd_avg_word_vectors(words, tfidf_vector, tfidf_vocabulary, model,
num_features):

    word_tfidfs = [tfidf_vector[0, tfidf_vocabulary.get(word)]
                    if tfidf_vocabulary.get(word)
                    else 0 for word in words]
    word_tfidf_map = {word:tfidf_val for word, tfidf_val in zip(words, word_
tfidfs)}

    feature_vector = np.zeros((num_features,),dtype="float64")
```

```

vocabulary = set(model.index2word)
wts = 0.
for word in words:
    if word in vocabulary:
        word_vector = model[word]
        weighted_word_vector = word_tfidf_map[word] * word_vector
        wts = wts + word_tfidf_map[word]
        feature_vector = np.add(feature_vector, weighted_word_vector)
if wts:
    feature_vector = np.divide(feature_vector, wts)

return feature_vector

# generalize above function for a corpus of documents
def tfidf_weighted_averaged_word_vectorizer(corpus, tfidf_vectors,
                                             tfidf_vocabulary, model, num_features):

    docs_tfids = [(doc, doc_tfidf)
                   for doc, doc_tfidf
                   in zip(corpus, tfidf_vectors)]
    features = [tfidf_wtd_avg_word_vectors(tokenized_sentence, tfidf, tfidf_
                                             vocabulary,
                                             model, num_features)
                for tokenized_sentence, tfidf in docs_tfids]
    return np.array(features)

```

The `tfidf_wtd_avg_word_vectors()` function helps us in getting the TF-IDF weighted averaged word vector representation for a document. We also create a corresponding generic function `tfidf_weighted_averaged_word_vectorizer()` to perform TF-IDF weighted averaging of word vectors for a corpus of documents. We can see our implemented function in action on our sample corpora using the following snippet:

```

# get tfidf weights and vocabulary from earlier results and compute result
In [453]: corpus_tfidf = tfidf_features
...: vocab = tfidf_vectorizer.vocabulary_
...: wt_tfidf_word_vec_features = tfidf_weighted_averaged_word_
vectorizer(corpus=TOKENIZED_CORPUS, tfidf_vectors=corpus_tfidf,
...:      tfidf_vocabulary=vocab, model=model,
...:      num_features=10)

```

```

...: print np.round(wt_tfidf_word_vec_features, 3)
[[ 0.011 -0.011  0.014 -0.011  0.007 -0.007 -0.024 -0.008 -0.004 -0.004]
 [ 0.     -0.014  0.028 -0.014  0.004 -0.003 -0.012  0.011 -0.001 -0.002]
 [-0.001 -0.008  0.007 -0.019  0.001 -0.004 -0.012 -0.018  0.001 -0.014]
 [-0.047  0.017 -0.012 -0.047 -0.044  0.002 -0.032 -0.034  0.027  0.03 ]]

# compute avgd word vector for test new_doc
In [454]: nd_wt_tfidf_word_vec_features = tfidf_weighted_averaged_word_
vectorizer(corpus=tokenized_new_doc, tfidf_vectors=nd_tfidf, tfidf_
vocabulary=vocab, model=model, num_features=10)
...: print np.round(nd_wt_tfidf_word_vec_features, 3)
[[-0.012 -0.019  0.018 -0.038 -0.006  0.01  -0.006 -0.011 -0.003  0.023]]

```

From the preceding results, you can see how we can converted each document into TF-IDF weighted averaged numeric vectors. We also used our TF-IDF weights and vocabulary, obtained earlier when we implemented TF-IDF-based feature vector extraction from documents.

Now you have a good grasp on how to extract features from text data that can be used for training a classifier.